LVGLUI: User Interface LVGL Library for NuttX

Apache Community Over Code 2026

13. 10. 2026

Michal Lenc, Martin Krasula, Michal Barták, Elektroline a.s.

Our devices and use cases

- Elektroline a.s.
- embedded devices running NuttX placed in panel boards, trolley buses, trams...
- most of them with LCD display
  - visualization of the current state
  - data visualization
  - configuration - menu with options



Our devices and use cases (ii)







LVGL library

- Light and Versatile Embedded UI Ecosystem

> Open-source graphics library for creating beautiful UIs on any MCU, MPU, and display type with a rich set of widgets, animations, and tools.
>
> — LVGL documentation

- MIT license
- object oriented library
- integrated support for NuttX's framebuffer and LCD drivers
- integrated to NuttX applications repository (`CONFIG_GRAPHICS_LVGL`)

LVGL UI

LVGL library (iii)

- https://gitlab.com/elektroline-predator/lvglui
- a library implementing UI pages and widgets used in Elektroline's devices
- the main idea is to provide common code base for the devices
- currently based on LVGL release v9.2.2
- MIT license
- uses LVGL implementation of NuttX LCD driver
- builds and runs on Linux under SDL2
- implements various useful features
  - prepared pages and layouts
  - display backlight intensity control
  - automatic display dim after timeout
  - data change save/revert
  - page's header colors based on status

Hardware Interfaces

Rotary encoder

- uses NuttX quadrature encoder driver and button driver
- moving between pages and properties by rotating the encoder
- encoder's button used for entering and leaving the page/property
- all LVGL UI pages/properties editable with this encoder

Exit button

- there might be another button on the device
- used as exit/reset button -> better user experience
- deselects the property or leaves the page
- returns to the main page if no page is entered
- uses NuttX button driver
- need to configure button indexes if used with encoder button

Hardware Interfaces (ii)

Keypad

- useful for bigger devices with more complicated configuration
- implemented as a custom device driver that returns 16 bit unsigned integer representing 4x4 matrix
- hard coded mapping to keys at the moment
- maps standard 0-9 numbers, enter and escape keys and three color buttons

Backlight

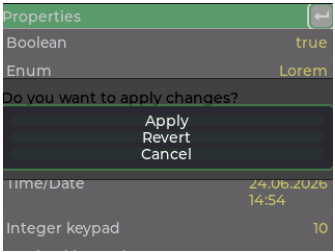
- support for GPIO (on/off) control or PWM output (controls intensity)

Pages

- UI that use the library typically consists of several pages
- typical design allows browsing through pages using the rotary encoder or keypad buttons
- page can be entered upon property click or event (multi-level pages)
- several prepared pages for typical use-case
- API to create a custom page

Property page

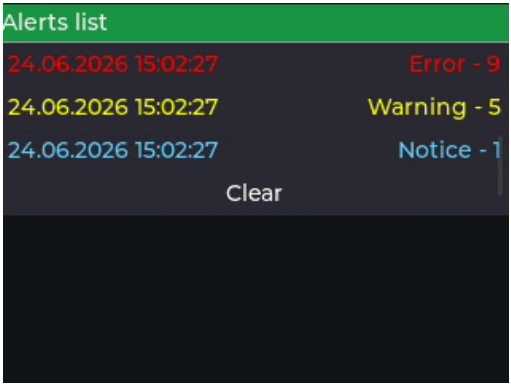
- various properties can be added to the page
- allows to view/edit properties and has a confirmation popup window to apply/ revert changes upon leaving
- optional callback upon page leave



Pages (ii)

Alerts page

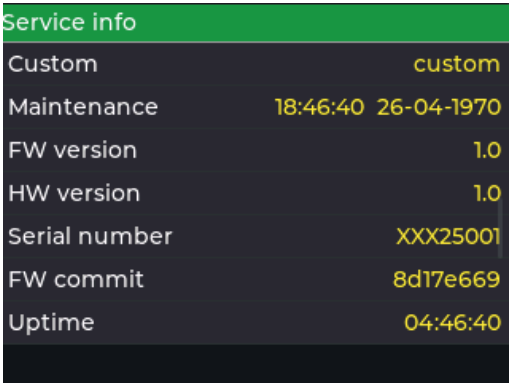
- special page for visualization of alerts
- support for three level of alerts - error, warning, notice
- every alert is a property with a description
- enterable page, has optional clear button
  - clears the alerts and calls user specific callback



Pages (iii)

Info page

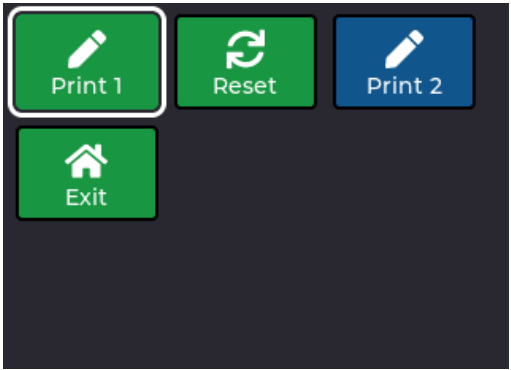
- designed to provide a read-only information about the device
- various predefined properties
  - FW version
  - HW version
  - Serial number
  - FW commit hash
- possibility to add custom string and integer



Pages (iv)

Grid page

- scrolling between a large number of pages is not user friendly
- grid page with “buttons” pointing to another page
- basically a menu page



Properties

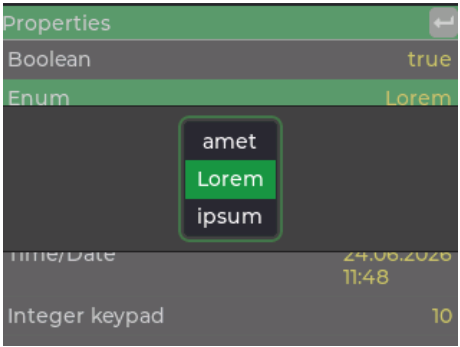
- prepared objects that can be placed on property page
- typically allows the user to read or set values
- device's configuration or control
- usually implemented with popup opened when clicked on the property

Boolean

- simple `True`/`False` property, click changes the value

Enum

- allows the user to chose from several options



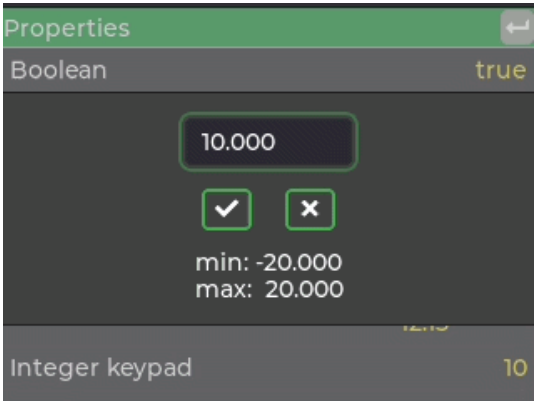
Properties (ii)

Integer

- allows to set 32 bit large signed integer value
- separate implementation for encoder and keypad

Decimal

- allows to set decimal value
- 64 bit signed mantissa, 32 bit signed exponent
- separate implementation for encoder and keypad



Properties (iii)

Double

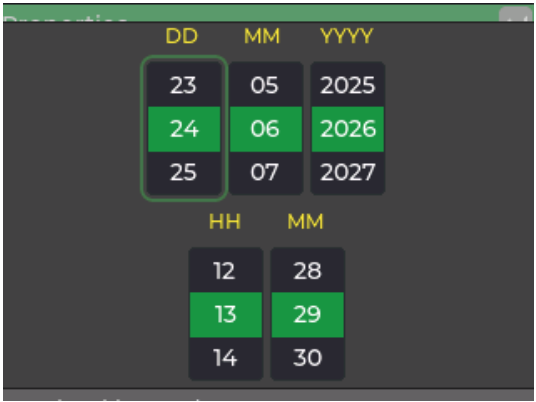
- read only property that can visualize double value in 64 bit format according to IEEE 754

String

- allows to modify a sequence of characters

Datetime

- allows to set the date and time with the precision up to seconds



API showcase


```c
#include <lvglui.h>

int main(int argc, char *argv[]) {
  lvglui_init();
  lvglui_loop();
  lv_deinit();
  return 0;
}
```

- `lvglui_init()` initializes the display, peripherals and calls `lv_init()` and `lvglui_content()` in which pages and properties are created
- `lvglui_loop()` runs in an infinite loop and processes input events
- the loop is terminated by calling `lvglui_loop_break()`

API showcase (ii)

Property page

```
void lvglui_content(void) {
  lvglui_props_t *props =
      lvglui_add_props_page("Properties", chng_callback);
  lvglui_prop_bool_t *bprop = lvglui_add_prop_bool(props, "Boolean", true);
  lvglui_prop_enum_t *eprop = lvglui_add_prop_enum(props, "Enum", true);
  lvglui_prop_int32_t *i32prop = lvglui_add_prop_int32(props, "Integer");

  lvglui_set_prop_bool(bprop, true);
  lvglui_set_prop_int32(i32prop, 10);
  lvglui_set_prop_int32_limits(i32prop, 0, 100);
}
```

API showcase (iii)

Grid page

```
static lvglui_grid_btn_t grid_btns[] = {
  {LV_SYMBOL_EDIT, "Print 1", grid_print, NULL},
  {LV_SYMBOL_REFRESH, "Reset", NULL, NULL},
  {LV_SYMBOL_EDIT, "Print 2", grid_print},
  {LV_SYMBOL_HOME, "Exit", grid_exit, NULL}, {NULL, NULL, NULL, NULL}
};

lvglui_grid_t *grid_page = lvglui_add_grid_page("Grid layout", grid_btns);
lvglui_set_grid_buttons(grid_page, grid_btns);
lvglui_set_grid_font(grid_page, &lv_font_montserrat_24, &lv_font_montserrat_14);
```

Demo application

- the repository provides a demo application with several pages and widgets
- you can use Nix to enter a reproducible development environment
- dependencies: Meson build system, SDL for Linux build, Cmake

```
$ nix develop
$ meson setup builddir
$ meson compile -C builddir
$ ./builddir/demo/lvgluidemo-landscape
```

- source file for demo is in `demo/main.c`

Memory usage

- LVGL can allocate the entire display buffer (width x height x bpp) unless configured otherwise
- we use partial rendering with 80 rows for 320 (w) x 240 (h) display
- configurable by Meson option in LVGL UI library

```
#define LV_USE_NUTTX_LCD      1
#if LV_USE_NUTTX_LCD
    #define LV_NUTTX_LCD_BUFFER_COUNT    0
    #define LV_NUTTX_LCD_BUFFER_SIZE     @LV_BUFFER_ROWS@
#endif

#define LV_USE_STDLIB_MALLOC    LV_STDLIB_CLIB
#define LV_USE_STDLIB_STRING    LV_STDLIB_CLIB
#define LV_USE_STDLIB_SPRINTF   LV_STDLIB_CLIB
```

Memory usage (ii)

Objects

- one boolean property (button and label objects) takes about 1 kB of RAM

> To work with lower LV_MEM_SIZE you can create objects only when required and delete them when they are not needed anymore
>
> — LVGL documentation

References

Come to NuttX Hackathon tomorrow from 11:20 AM to 14:10 PM.

- LVGL UI repository
- LVGL